

智慧冷链 / 仓储环境监控系统

coldchain_monitor

目标平台	GEC6818 ARM Cortex-A53
显示器	800 × 480 LCD (/dev/fb0, 32-bit RGBA)
传感器	STM32 节点 → UART2 (/dev/ttySAC3, 115200 bps)
编译器	arm-linux-gcc (交叉编译)
依赖库	pthread / libm / stb_truetype

安徽中科康芯 嵌入式 Linux 学习项目

2026-03-09 / 2026-03-10

1 项目概述

coldchain_monitor 是一个运行于 GEC6818 ARM Cortex-A53 开发板上的嵌入式 Linux 应用程序。它通过串口 (UART) 实时接收 STM32 传感器节点上报的冷链环境数据，并利用 Linux Framebuffer 驱动将温度、湿度、门状态、报警状态、气味状态及当前时间实时渲染到 800×480 LCD 屏幕上，适用于冷库 / 冷链运输的现场监控场景。

- 实时采集：串口每帧数据到达后自动校验并刷新界面
- 多线程架构：时间线程与传感器线程并发，互不阻塞
- TrueType 渲染：使用 stb_truetype 库渲染仿宋中文字体
- 纯 Linux C 实现：无 Qt / LVGL 等 GUI 框架依赖，原理清晰

2 硬件环境

部件	型号 / 说明
开发板	GEC6818 (Samsung S5P6818, ARM Cortex-A53 × 8, 1.4 GHz)
显示屏	800 × 480 TFT-LCD, /dev/fb0, 32-bit RGBA (每像素 4 字节)
传感器节点	STM32F103, 挂载 DHT22 (温湿度) + 门磁 + MQ 气体传感器
串口连接	STM32 UART2 → GEC6818 /dev/ttySAC3, 波特率 115200 bps, 8N1
字体文件	simfang.ttf (仿宋, 需与可执行文件放在同一目录)

3 工程文件结构

```
coldchain_monitor/  
■■■■ Makefile                # ■■■■■■  
■■■■ main.c                  # ■■■■■■LCD■■■■■■■■■■■■■■■■■■■■  
■■■■ inc/  
■ ■■■■ coldchain_monitor.h   # ■■■■■■■■■■■■■■■■■■■■  
■ ■■■■ font.h                 # ■■■/■■■■■■■■■■  
■ ■■■■ truetype.h             # stb_truetype ■■■■  
■ ■■■■ lcd.h                  # LCD BMP ■■■■■■  
■ ■■■■ bitmap.h              # ■■■■■■  
■■■■ src/  
    ■■■■ coldchain_monitor.c  # ■■■■■■LCD■■■■■■■■■■■■■■■■■■■■  
    ■■■■ font.c               # FreeType■■■■UTF-8→UCS2■■■■■■  
    ■■■■ truetype.c           # stb_truetype ■■■■  
    ■■■■ lcd.c                # BMP■■■■■■■■■■
```

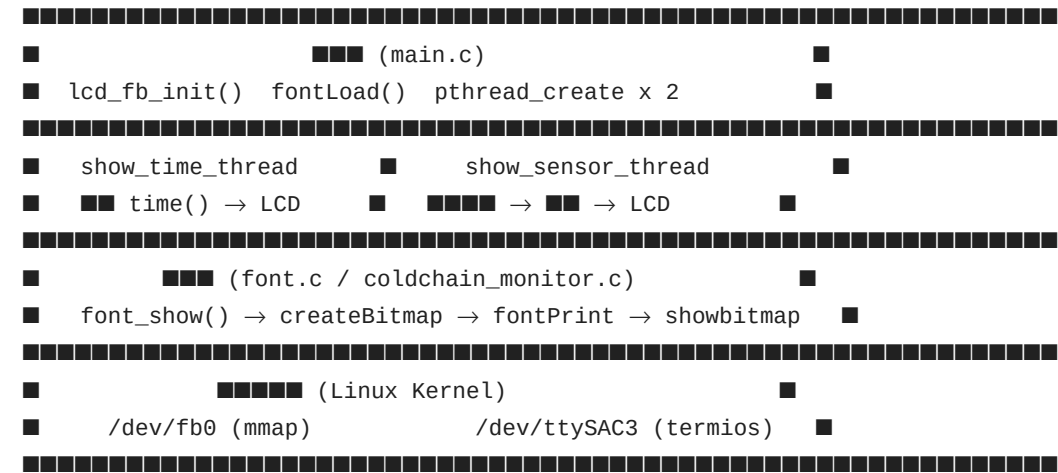
文件	职责
main.c	初始化 Framebuffer、加载字体、创建并等待两个 pthread
coldchain_monitor.c	LCD驱动(mmap)、font_show渲染、串口配置、帧状态机解析、界面布局

font.c	UTF-8→UCS2 转换，调用 stb_truetype 光栅化并 alpha 混合到 bitmap
truetype.c	stb_truetype 单头库的 C 实现（矢量字体核心）
lcd.c	从文件读取 BMP 并显示到 Framebuffer（备用功能）

4 系统架构与线程模型

4.1 总体架构

程序分为三层：硬件驱动层（/dev/fb0 + /dev/ttySAC3）、中间层（字体渲染 + 帧解析）、应用层（界面布局 + 线程调度）。



4.2 线程设计

线程	函数	周期	职责	互斥锁
主线程	main()	一次性	初始化 LCD/字体，创建子线程，阻塞 join—	
时间线程	show_time_thread()	1 秒	读取系统时间，调用 font_show 刷新右上角时间区域	lcd_mutex
传感器线程	show_sensor_thread()	2秒超时	阻塞读串口字节，状态机组帧，XOR校验，刷新传感器数据	lcd_mutex, sensor_mutex

[提示] 两个线程共享 lcd_mutex，保证 showbitmap 写显存时不会互相撕裂画面。

5 串口数据帧格式（11 字节）

STM32 每次采样后通过 UART 发送 11 字节定长帧，GEC6818 以状态机逐字节接收并校验。

偏移	字段	长度	说明
[0]	帧头1	1 B	固定 0xAA
[1]	帧头2	1 B	固定 0x55
[2]	温度高字节	1 B	有符号 16-bit 整数高 8 位，÷10.0 = °C
[3]	温度低字节	1 B	有符号 16-bit 整数低 8 位

[4]	湿度高字节	1 B	无符号 16-bit 整数高 8 位, ÷10.0 = %RH
[5]	湿度低字节	1 B	无符号 16-bit 整数低 8 位
[6]	门状态	1 B	0 = 关闭 1 = 开启
[7]	报警状态	1 B	0 = 安全 1 = 报警
[8]	气味状态	1 B	0 = 正常 1 = 异常
[9]	XOR 校验	1 B	[2]^[3]^[4]^[5]^[6]^[7]^[8]
[10]	帧尾	1 B	固定 0x0D

5.1 帧解析状态机

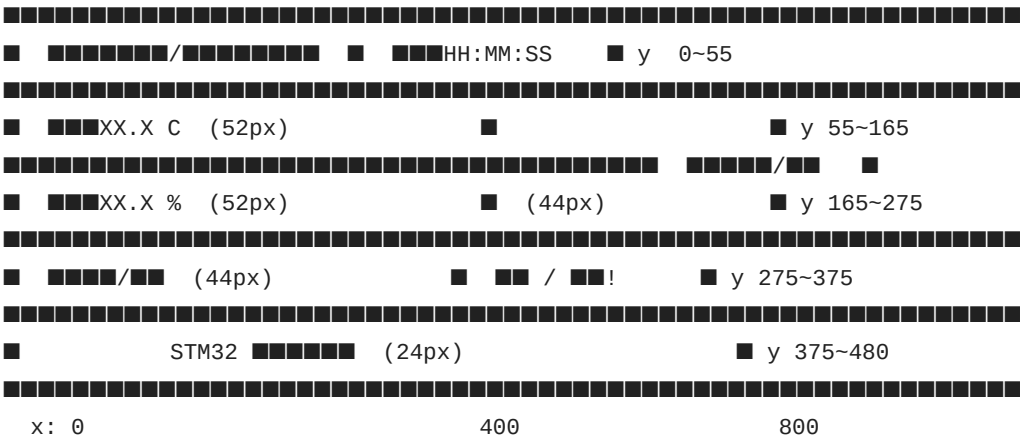
```
state=0  ■■■■1 (0xAA)
    byte==0xAA → frame[0]=byte, state=1

state=1  ■■■■2 (0x55)
    byte==0x55 → frame[1]=byte, idx=2, state=2
    ■■        → state=0 (■■■■)

state=2  ■■■■ 9 ■■ (idx: 2 → 10)
    idx < FRAME_LEN → frame[idx++] = byte
    idx == FRAME_LEN → ■■■■ & XOR → ■■■■ → ■■■■
                      state=0 (■■■■■)
```

[注] 若 read() 超时返回 0 (2秒无数据) , 线程用上次采集值重新刷新界面, 防止串口中断后 LCD 黑屏。

6 LCD 屏幕布局 (800 × 480)



区域	坐标范围	内容	字号(px)	文字颜色
标题 (左)	x:0-400 y:0-55	智慧冷链/仓储环境监控	32	黄 #FFFF00
时间 (右)	x:400-800 y:0-55	时间: HH:MM:SS	28	红 #FF0000
温度 (左)	x:0-400 y:55-165	温度: XX.X C	52	黄 #FFFF00

湿度 (左)	x:0-400 y:165-275	湿度: XX.X %	52	黄 #FFFF00
气味 (右)	x:400-800 y:55-275	气味: 正常/异常	44	黄 #FFFF00
门 (左)	x:0-400 y:275-375	门: 关闭/开启	44	红/蓝(状态)
报警 (右)	x:400-800 y:275-375	安全 / 报警!	44	红/蓝(状态)
底栏 (全)	x:0-800 y:375-480	STM32 冷链监控节点	24	橄榄 #888800

7 关键代码解析

7.1 LCD Framebuffer 初始化 (lcd fb init)

通过 mmap 将 /dev/fb0 映射到进程地址空间，写显存等同于写像素：

```
lcd_fd = open("/dev/fb0", O_RDWR);

lcd_map = mmap(NULL,
                SCREEN_H * SCREEN_W * SCREEN_PIXEL_SIZE, // 800*480*4 = 1.5 MB
                PROT_READ | PROT_WRITE,
                MAP_SHARED, lcd_fd, 0);

// ■■■■(x,y) → lcd_map[y*800*4 + x*4 + ch]
// ch: 0=B 1=G 2=R 3=A (BGRA 32-bit)
```

7.2 字体渲染流水线 (font show)

font show 是界面绘制的统一入口，每次创建离屏 bitmap 再贴到 LCD:

```
void font_show(char* text, int pixelSize,
               int outW, int outH, int bgColor,
               int fontX, int fontY, int fontColor,
               int lcdX, int lcdY)
{
    pthread_mutex_lock(&lcd_mutex);           // ■■■■
    fontSetSize(my_font, pixelSize);          // ■■■■
    bitmap* bm = createBitmapWithInit(        // ■■■■■■■■■■■■■■■■■■■■■■
        outW, outH, 4, bgColor);
    fontPrint(my_font, bm, fontX, fontY,      // UTF-8→UCS2→■■■■→alpha■■■■
        text, fontColor, 0);
    showbitmap(bm, lcdX, lcdY);                // ■■■■■■ lcd_map
    pthread_mutex_unlock(&lcd_mutex);
    destroyBitmap(bm);                         // ■■■■■■
}
```

7.3 串口初始化 (termios)

使用 POSIX termios API 将串口配置为原始模式, 115200 bps, 8N1:

```
struct termios t;
bzero(&t, sizeof(t));
cfmakeraw(&t); // ██████████
cfsetispeed(&t, B115200); // ████████
cfsetospeed(&t, B115200); // ████████
```

```

t.c_cflag |= CS8;           // 8 位
t.c_cflag &= ~PARENB;      // 无奇偶校验
t.c_cflag &= ~CSTOPB;      // 1 位
t.c_cc[VMIN] = 0;          // 无最小字符数
t.c_cc[VTIME] = 20;        // 2 秒 0.1s
tcsetattr(fd, TCSANOW, &t);

```

7.4 全局变量与互斥锁

```

// 全局变量
double g_temp, g_humi;
int g_door, g_alarm, g_smell;
pthread_mutex_t sensor_mutex = PTHREAD_MUTEX_INITIALIZER;

// LCD 全局变量
pthread_mutex_t lcd_mutex = PTHREAD_MUTEX_INITIALIZER;

```

[提示] sensor_mutex 保护传感器数据写入；lcd_mutex 保护显存写入。两者独立，避免死锁。

8 TrueType 字体渲染原理

项目使用 stb_truetype（单文件 C 库）进行矢量字体光栅化，font.c 封装为中文友好 API：

- fontLoad(): 读取 ttf 文件到内存，stbtt_InitFont 建立字形索引
- fontSetSize(): stbtt_ScaleForPixelHeight 计算缩放比例
- utf8_to_ucs2(): UTF-8 编码字符串 → UCS-2 宽字符数组
- fontPrint(): 遍历 UCS-2 字符 → stbtt_MakeCodepointBitmap 光栅化 → alpha 混合写入 bitmap
- showbitmap(): 将 bitmap.map 逐像素拷贝到 lcd_map（显存）

Alpha 混合公式（实现抗锯齿）：

```

// charRaster[i] = alpha (0~255)
r_out = (alpha * fontR + (255 - alpha) * bgR) >> 8;
g_out = (alpha * fontG + (255 - alpha) * bgG) >> 8;
b_out = (alpha * fontB + (255 - alpha) * bgB) >> 8;

```

9 编译与部署

9.1 交叉编译 (Ubuntu / WSL)

```

# 编译 Makefile
cd coldchain_monitor
make

# 交叉编译
arm-linux-gcc -Wall -I inc/ -o coldchain_monitor \
    main.c src/coldchain_monitor.c src/font.c src/lcd.c src/truetype.c \
    -lpthread -lm

```

9.2 部署到开发板

```
# █ █ NFS / TFTP / SCP █ █ █ █
scp coldchain_monitor simfang.ttf root@192.168.x.x:/home/

# █ █ █ █ █ █ █ █
chmod +x coldchain_monitor
./coldchain_monitor
```

[注] simfang.ttf 必须与 coldchain_monitor 可执行文件在同一目录，否则字体加载失败程序退出。

依赖项	检查命令	说明
/dev/fb0	ls -la /dev/fb0	LCD Framebuffer, 内核已加载 fbdev 驱动
/dev/ttySAC3	ls -la /dev/ttySAC3	串口设备, STM32 连接后自动出现
simfang.ttf	ls simfang.ttf	仿宋字体文件, 与程序同目录
pthread/libm	arm-linux-gcc -v	工具链内置, 无需额外安装

10 数据流总结

```
STM32 █ █ █ █
|   █ █ DHT22/█ █ /MQ → █ █ 11█ █ █ █ → UART2 █ █
|
/dev/ttySAC3 (115200 bps)
|
|   show_sensor_thread  read() █ █ █ █
█ █ █ █ (state 0→1→2) █ █ █ █
|   █ █ █ █ + XOR █ █ █ █
|
█ █ █ █ █ █ █ █
temp = (int16_t)(frame[2]<<8 | frame[3]) / 10.0
humi = (uint16_t)(frame[4]<<8 | frame[5]) / 10.0
door = frame[6], alarm = frame[7], smell = frame[8]
|
|   pthread_mutex_lock(&sensor_mutex)
█ █ █ █ █ █ █ █ g_temp / g_humi / g_door / g_alarm / g_smell
|
|   sensor_display_all()
font_show × 7 → createBitmapWithInit → fontPrint → showbitmap
|
|   pthread_mutex_lock(&lcd_mutex)
█ █ lcd_map█ mmap █ █ █ █ → LCD █ █ █ █ █ █ █ █
```